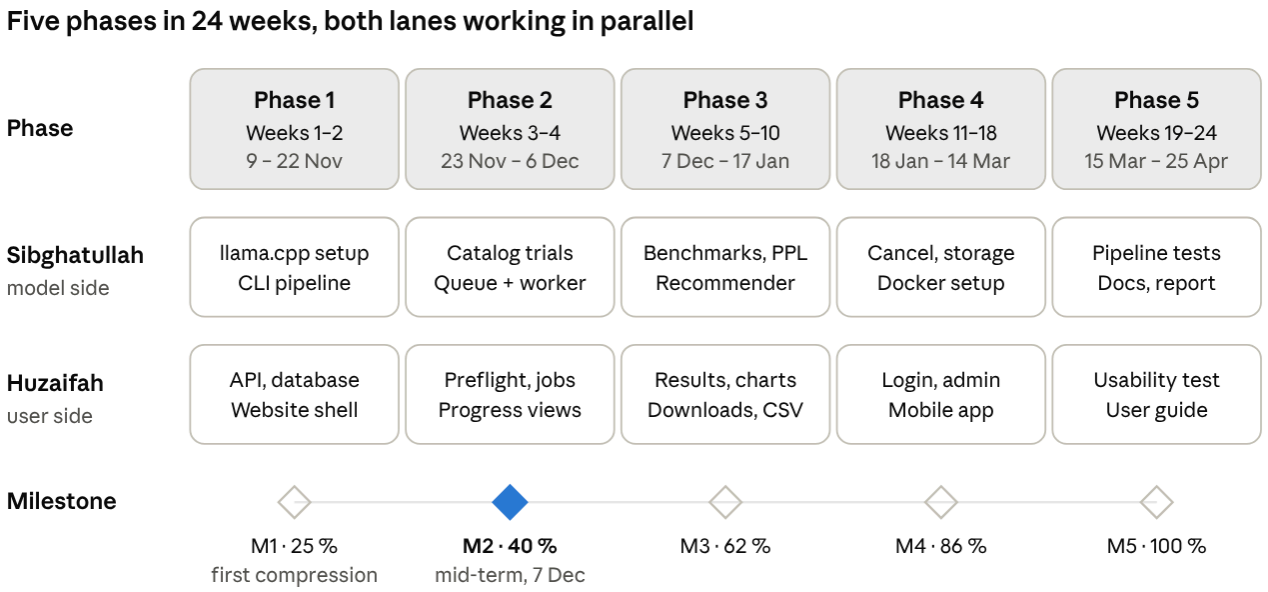
PocketPrune – Complete Implementation Roadmap (100%)

Nov 9, 2026 · @Sibhuz

The whole system — pipeline, evaluation, recommender, website and mobile app — is built in 24 weeks (9 Nov 2026 to 25 Apr 2027), with both members working in parallel from week 1. **40 % of the project must be shown at the mid-term on 7 December 2026**, so Phases 1 and 2 are packed into the first four weeks; the remaining 60 % is spread over the rest of the semester.

At a glance



24-week roadmap · 5 phases, 2 lanes, milestones M1–M5

Each milestone shows how much of the whole project is done when it is reached: M1 25 %, **M2 40 % (mid-term, 7 Dec)**, M3 62 %, M4 86 %, M5 100 %.

What to build first, and what later

You're new to model compression, so the order is: **learn → prove compression works → connect it end to end → measure and recommend → complete the product → polish**.

1. **Prove the risky part first.** Before any job pages, Sibghatullah compresses a tiny model by hand, then turns it into a command-line pipeline. Once llama.cpp works on your machine, the rest is normal web development — and the mid-term demo shows a real compression, not mock-ups.

2. **One thin path from the website, end to end.** Job Setup → preflight → job → worker → files → live progress, with plain pages. No charts, login or admin yet.
3. **Measure, compare, recommend.** Real numbers against F16, explainable rules checked against baselines, and file downloads — the part that makes this an FYP rather than a wrapper.
4. **Complete the product.** Login, history, cancel/retry, clean-up, admin, help, Docker — then the mobile app if you choose.
5. **Prove and present it.** Tests, security check, usability test, optional hosting, guides.

Priority	What	When
Must-have	Catalog, preflight, pipeline, worker, live progress, benchmarks, perplexity, recommender, results, downloads	Phases 1–3
Should-have	Login, history, cancel/retry, storage clean-up and limits, admin, help, Docker, mobile app, tests, guides	Phases 4–5
Could-have	Free hosting, "Publish to Hugging Face", a "compare outputs" box (type a prompt, see F16 vs Q4 answers side by side), Q8_0, gated Llama-3.2, Phi-3.5-mini	Only after M4 is safe

How you'll work

Sibghatullah owns the **model side** (llama.cpp, pipeline, job worker, evaluation, recommender) and Huzaifah owns the **user side** (MongoDB models, Express API, website, and later the app). The stack is **MERN in plain JavaScript (JS/JSX)** — MongoDB, Express, React, Node.js — plus llama.cpp for the model work; code is kept simple enough for both of you to explain in the viva. **The website is built first; the mobile app starts only after the web app is complete (Phase 4).** Both lanes run in parallel from week 1, and the milestones M1–M5 stay as in the proposal. The full build spec is in `POCKETPRUNE_BUILD_PROMPT.md`.

Area	Owner	Stack
llama.cpp build, catalog trials	Sibghatullah	llama.cpp (pinned release tag) on Ubuntu 22.04 / WSL2; Python venv only for its conversion script and `hf download`

Area	Owner	Stack
Worker and pipeline	Sibghatullah	Separate Node.js process in `backend/worker/` that picks the oldest queued job from MongoDB; `pidusage` for RAM
Evaluation and recommender	Sibghatullah	llama-bench, llama-perplexity, local WikiText-2; recommender as plain functions in `backend/services/recommender.js`
Database and REST API	Huzaifah	MongoDB + Mongoose, Express, bcryptjs + jsonwebtoken, express-rate-limit, morgan
Website	Huzaifah	React (Vite, JSX), React Router, axios, Recharts; styling library of your choice, with all colours and spacing in one `theme.js` that the app copies
Mobile app (Phase 4, after the website)	Huzaifah (Sibghatullah helps)	React Native + Expo (JavaScript), same REST API
Tests, CI, Docker	Both	Vitest, supertest, mongodb-memory-server, GitHub Actions, Docker Compose
Docs, SRS, report, slides	Both	`docs/PROJECT_STATUS.md`, `docs/DEV_LOG.md`, Overleaf, department templates

Repository (one GitHub repo):

```
pocketprune/
  package.json      root scripts: install-all, dev (backend + worker +
frontend), seed, test
  backend/          config/ models/ controllers/ routes/ middleware/
services/ worker/ utils/
                    server.js, seed.js, cli.js, tests/
  frontend/         src/pages/ admin/ components/ context/ services/api.js
utils/ styles/
  scripts/          build-llamacpp.sh, fetch-wikitext.js, validate-
catalog.js, regression.js
  docs/             PROJECT_STATUS.md, DEV_LOG.md, HOW_IT_WORKS.md,
GIT_GUIDE.md, API.md, SRS
  deploy/           docker-compose.yml, optional hosting files
  mobile/           created later, when the app starts
  data/             models, artifacts, wikitext (git-ignored)
```

Build order rule: the pipeline always comes before the page that shows it. When the pipeline for a feature isn't ready, Huzaifah builds the page with fake data first, then switches to the real endpoint. **Documentation rule:** at the end of every session, update `docs/PROJECT_STATUS.md` (what's done, what's left) and add an entry to `docs/DEV_LOG.md`, so either of you can pick up where the other stopped.

Machines: Sibghatullah's laptop is the **model machine** — Ubuntu via WSL2, llama.cpp, the worker, and every measurement in the report. Huzaifah stays on **plain Windows**: the API, website and app need no WSL2 or llama.cpp. Both connect to one free **MongoDB Atlas** database, so jobs created on Huzaifah's website are run by Sibghatullah's worker and the real results show up on both. Until the real pipeline is finished, a temporary **fake pipeline** (built in WP-06, deleted in WP-20) lets the website be built and tested on Windows with realistic sample numbers, clearly marked "FAKE DATA".

Model files: the compressed `.gguf` files are never stored in MongoDB — the database keeps only each file's path, size, checksum and measurements (a few KB per job). Files live on the server's disk and are streamed when someone clicks Download. Disk stays under control because the original download is deleted after conversion and the F16 file after evaluation, identical requests reuse an existing file, files expire after 7 days unless pinned, and each user has a 3 GB storage limit. Typical Q4 sizes: 0.1 GB (SmolLM2-135M) to about 1 GB (1.5B models). The app never downloads model files; it links to the website.

GitHub workflow (counts toward the marks):

- The work is split into **33 work packages (WP-00 to WP-32)**, plus four mobile packages (WP-M1 to WP-M4) that start only when you decide. Owners alternate, so both of you open pull requests in every phase — about 16 each across the semester.

- One work package = one GitHub issue, one branch (e.g. `feat/wp09-pipeline-stages`) and one pull request. Commit locally at each working step (4–10 commits), **push once** when the package is done.
- Build in steps: finish, review and merge a package before starting the next one that depends on it. Never build several packages and push them together — aim for 1–2 PRs per person per week.
- The other member reviews (with at least one real comment or approval note) and merges; nobody pushes to `main`. Merges use "Create a merge commit" so each branch stays visible.
- From Phase 5 on, small fixes are batched into one fix PR every few days (`fix/frontend-polish-wk21`); only blocking bugs get an immediate PR.
- Each milestone ends with a tag and GitHub release: `v0.1.0-M1`, `v0.2.0-M2`, `v0.3.0-M3`, `v0.4.0-M4`, `v1.0.0` (and `v1.1.0-mobile` if the app is released separately).
- Rhythm: Monday 30-minute planning, 10-minute daily check-in, short demo to Ms. Hafsa Tahir / Ms. Unaisa Batool every 2 weeks.

Progress tracking and staying in sync

Every work package has a **weight** — its share of the whole project — and the weights add up to 100 %. The project percentage counts only **merged** work packages, so the number is honest: code on a branch or waiting for review doesn't count yet.

Phase	Work packages	Weight	Project done	Target
1 — Learn, set up, first compression	WP-00 – WP-07	25 %	25 %	22 Nov
2 — First full job from the website	WP-08 – WP-13	15 %	**40 %**	**6 Dec (mid-term 7 Dec)**
3 — Measure, compare, recommend	WP-14 – WP-20	22 %	62 %	17 Jan
4 — Complete product + mobile app	WP-21 – WP-26, WP-M1 – WP-M4	24 %	86 %	14 Mar
5 — Testing and delivery	WP-27 – WP-32	14 %	100 %	25 Apr

The split is even: about 50 % of the weight each.

How it works day to day:

- `docs/progress.json` lists every work package with its weight, owner, week, status and the packages it needs first. Running `npm run progress` prints the overall percentage, each phase, each person's share, the days left to the next target, and who is waiting on whom.
- **Before starting a package,** the coding agent checks its "Needs". If the other person's package isn't merged yet, it says so — e.g. *"WP-12 needs Sibghatullah's WP-11. Meanwhile do WP-08."*
- **After a package is merged,** the agent reports the new percentage, what it unblocks for the other person, and a ready-to-send message — e.g. *"✅ WP-10 merged — 34 %. This unblocks WP-11 for Sibghatullah: 'Jobs API is on main, pull it and start the worker.'"*
- **Phase gates:** when every package in a phase is merged, the agent announces it (*"Phase 2 complete — 40 % reached"*), reminds you to tag the release, and names each person's first package in the next phase.
- **Monday sync (15 minutes):** both run `npm run progress`, agree on the week's packages and clear anything blocked.
- **Sprint to 40 %:** in weeks 1–4 each of you opens about 3 pull requests a week; after the mid-term, about 1 a week.

Phase 1 — Learn, set up, first real compression (weeks 1–2, 9–22 Nov) → 25 %

Goal: llama.cpp working on the model machine, a tiny model compressed by hand and then by a command-line pipeline (plus the temporary fake pipeline), the backend core, and a first website showing the catalog.

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
1 (9 Nov)	**WP-02 (4 %):** set up the model machine (WSL2 + Ubuntu, Node 20); build llama.cpp at one release tag with a Python venv; compress SmolLM2-135M **by hand**; explain each command in `LEARNING_NOTES.md`; open the PR	**WP-00 (1 %)** with Sibghatullah: GitHub repo, labels, milestones, issues, branch protection, shared MongoDB Atlas database. **WP-01 (3 %):** folders, root scripts incl. `npm run progress`, `progress.json`, skeletons, docs, templates, CI; open the PR. **WP-03 (3 %) starts**	WP-00, WP-01, WP-02 merged → **8 %**

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
2 (16 Nov)	**WP-04 (3 %):** `runCommand.js`, `llamacpp.js`, `downloader.js` with mocked tests; open the PR. **WP-06 (5 %) starts:** `pipeline.js`, `cli.js`, temporary `fakePipeline.js`	WP-03: models, error codes, demo-user login, health endpoints; open the PR. **WP-05 (3 %):** catalog and profile API, `seed.js`; open the PR. **WP-07 (3 %) starts:** styling choice, `theme.js`, base components	WP-03, WP-04, WP-05 merged → **17 %**

WP-06 and WP-07 are finished at the start of week 3; when both merge, **M1 = 25 %** — tag `v0.1.0-M1` (target 22 Nov, latest 25 Nov).

REQs completed: REQ-1 to REQ-4; REQ-11 to REQ-14 working from the command line.

Phase 2 — First full job from the website (weeks 3–4, 23 Nov – 6 Dec) → 40 % for the mid-term

Goal: one thin, working path — a job set up on the website is preflighted, queued and run by the worker into validated Q5_K_M and Q4_K_M files, with live progress. Plain pages; no charts, login or admin yet. **Mid-term demo (Monday 7 Dec):** pick SmolLM2-135M on the website → preflight checklist → submit → live progress while the real worker compresses it → two valid files; then the SRS and slides.

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
3 (23 Nov)	Finish WP-06; open the PR. **WP-09 (2 %):** try the other catalog models, `catalog-trials.md`, `validate-catalog.js`, `fetch-wikitext.js`; open the PR	Finish WP-07; open the PR. **WP-08 (3 %):** preflight service and endpoint, real checklist on Job Setup; open the PR. **WP-10 (3 %) starts:** jobs API	**M1 tagged (25 %)**; WP-08 and WP-09 merged → **30 %**

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
4 (30 Nov)	**WP-11 (3 %)** Mon–Wed: worker — picks queued jobs, runs the pipeline (or the fake one on Windows), heartbeat, cancel flag; open the PR. **WP-13 (2 %)** Thu–Sat: time limits, clean-up, disk reserve, `regression.js`; open the PR. SRS into LaTeX on Overleaf	WP-10 PR on Monday (Sibghatullah's worker needs it). **WP-12 (2 %):** Job Progress page — stepper, bar, log panel, "FAKE DATA" badge; open the PR. Slides on the department template	All Phase 2 work merged → **40 %**; tag `v0.2.0-M2` on 6 Dec

If time runs short, WP-13 is the one to finish last — the demo works without it at 38 %.

REQs completed: REQ-6 to REQ-16; REQ-26 started.

Phase 3 — Measure, compare, recommend (weeks 5–10, 7 Dec – 17 Jan) → 62 %

Goal: every completed job shows measured size, speed, memory and perplexity against F16, and a recommendation with its reason, on the website.

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
5–6 (7 Dec)	**WP-14 (4 %):** `evaluator.js` — size, `llama-bench` JSON, 3 runs averaged, unstable flag, peak RAM; open the PR	**WP-15 (3 %):** results endpoint and Results comparison table; open the PR	Real numbers on the website → **47 %**
7 (21 Dec)	**WP-16 (3 %):** perplexity, change vs F16, saved settings, demo mode, delete the F16 file after evaluation; open the PR	**WP-17 (3 %):** three bar charts, Demo badge, machine name; Downloads page and download endpoint; open the PR	All metrics and charts; a GGUF downloads → **53 %**

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
8–9 (28 Dec)	**WP-18 (4 %):** `recommender.js` — fits, quality guard, preference ranking, tie-break, nothing-fits message, baselines, explanation; one test per rule; open the PR	**WP-19 (3 %):** recommendation card, recommend endpoint, profile dropdown, CSV/JSON export; open the PR	Changing the profile changes the recommendation → **60 %**
10 (11 Jan)	**WP-20 (2 %):** full catalog run, repeat one job to check results match, `benchmark-report.md`; **delete the fake pipeline** and its badge; open the PR	Review WP-20; small fixes; screenshots for the report	**M3 = 62 %**; tag `v0.3.0-M3`

REQs completed: REQ-17 to REQ-25, REQ-28, REQ-29; downloads part of REQ-30.

Phase 4 — Complete product + mobile app (weeks 11–18, 18 Jan – 14 Mar) → 86 %

Goal: a finished multi-user product — accounts, job control, history, storage clean-up and limits, administration, help and Docker — and then the mobile app on the same API (started when you decide; weeks 15–18 are its slot).

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
11 (18 Jan)	**WP-22 (3 %) starts:** cancel — stop the running command, delete half-written files; review WP-21	**WP-21 (3 %):** register/login, real `protect`, admin guard, own data only, Login/Register pages; open the PR	No token → 401 → **65 %**
12 (25 Jan)	WP-22: retry reusing downloads; open the PR	**WP-23 (3 %) starts:** History page with filters	Cancel and retry work → **68 %**

Week	Sibghatullah (model side)	Huzaifah (user side)	Exit check
13 (1 Feb)	**WP-24 (2 %):** `storage.js` — 7-day expiry + pin, **reuse identical files**, per-user usage, catalog validation job; open the PR	WP-23: Cancel/Retry buttons; open the PR	A repeated job reuses files → **73 %**
14 (8 Feb)	**WP-26 (2 %) starts:** `docker-compose.yml`	**WP-25 (3 %):** Admin pages, Help page, **per-user storage limit** in preflight and on Downloads; open the PR	Admin blocked for normal users; limit enforced → **76 %**
15 (15 Feb)	WP-26: integration check, load test, `performance.md`; open the PR	**WP-M1 (2 %):** Expo app setup, copied theme and components, Login	`docker compose up` works offline → **80 %**
16 (22 Feb)	**WP-M2 (2 %):** Catalog, New Job and Progress screens	Review WP-M2; optional: deploy the website to Vercel	Job started on the phone runs → **82 %**
17 (1 Mar)	Review WP-M3; help with app testing	**WP-M3 (2 %):** Results, History and Settings screens	App shows results → **84 %**
18 (8 Mar)	**WP-M4 (2 %):** APK build with Expo EAS, `mobile/README.md`	Batched fix PR; UI polish; demo to supervisors	**M4 = 86 %**; tag `v0.4.0-M4`

REQs completed: REQ-5, REQ-26, REQ-27, REQ-30 to REQ-33 — all 33 functional requirements done.

Phase 5 — Testing, documentation and final demo (weeks 19–24, 15 Mar – 25 Apr) → 100 %

Goal: a tested, documented, packaged system and a confident final demonstration. No new features after week 18.

Week	Sibghatullah	Huzaifah	Exit check
19 (15 Mar)	**WP-27 (3 %):** worker and pipeline failure-case tests, coverage report; open the PR	**WP-28 (3 %):** API failure tests, `docs/security.md`; open the PR	Backend coverage ≥ 60 % → **92 %**

Week	Sibghatullah	Huzaifah	Exit check
20–21 (22 Mar)	**WP-30 (2 %):** optional free hosting — or a polished offline Docker demo; open the PR	**WP-29 (2 %):** usability test with 5+ classmates (SUS) and fixes; open the PR	Demo setup works end to end → **96 %**
22 (5 Apr)	**WP-32 (2 %):** install/admin guide, benchmark methodology; open the PR	**WP-31 (2 %):** user guide, demo script, demo video; open the PR	All guides in `docs/` → **100 %**
23 (12 Apr)	Batched fix PR; rehearse the laptop fallback	Batched fix PR; final report chapters	Everything merged
24 (19 Apr)	Final report chapters on pipeline and results; rehearse	Final report chapters on design and UI; slides; tag `v1.0.0`	**M5:** final demo

Final showcase: free hosting and quick demo models

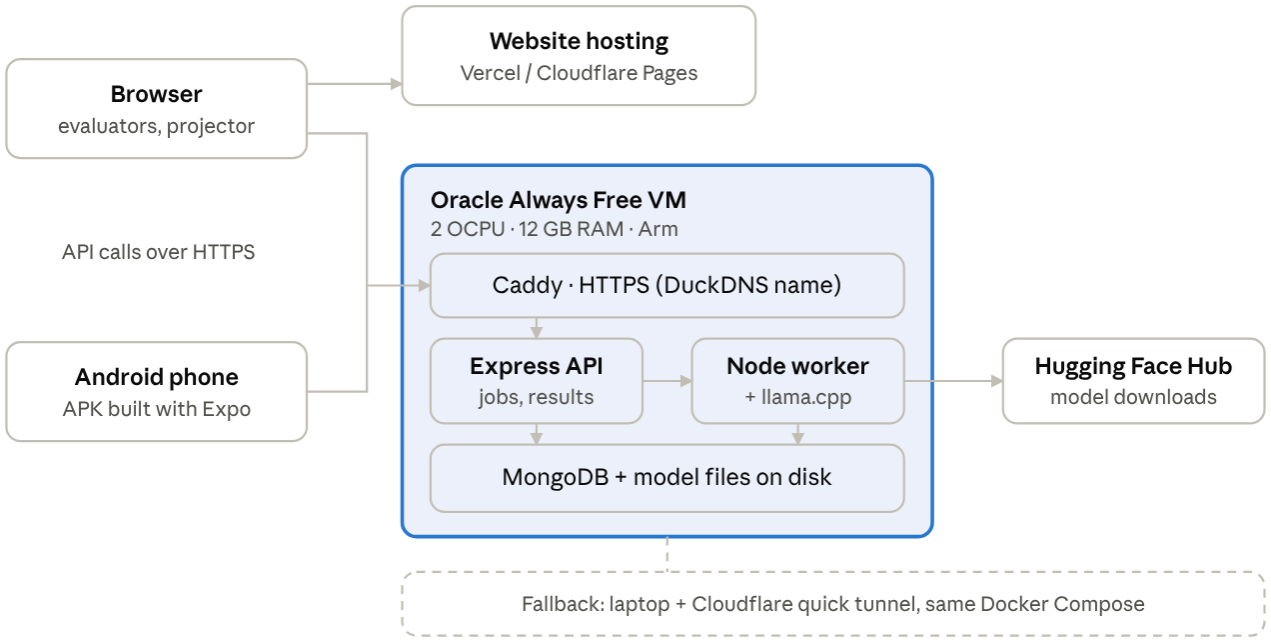
If hosting works out, PocketPrune runs online at zero cost for the final evaluation; if not, the same Docker Compose setup runs offline on a laptop over the local network, and every live job uses a tiny model that finishes in about 2–3 minutes. Larger models are processed in advance, so their results are already in History when the evaluators arrive.

Free hosting plan (checked November 2026)

Part	Free service	Why this one	Limits to remember
Express API + Node worker + llama.cpp + MongoDB	Oracle Cloud Always Free Ampere A1 VM (Arm), running the Docker Compose stack	The only free option with enough RAM for llama.cpp: 2 OCPU and 12 GB RAM, with block storage	Card needed for identity check; idle accounts can be reclaimed; free Arm capacity is sometimes "out of capacity" — create the VM by week 19
Database (alternative)	MongoDB Atlas free cluster	Keeps the database off the VM if you prefer	Small storage; job events can grow, so keep retention on

Part	Free service	Why this one	Limits to remember
HTTPS for the API	Caddy on the VM + a free DuckDNS subdomain	The website is served over HTTPS, so the API must be HTTPS too	Certificates renew automatically
Website	Vercel Hobby or Cloudflare Pages	The React build is static; deploys from GitHub on every merge to `main`	Hobby plan is for non-commercial use, which fits an FYP
Android app	Expo free plan: EAS Build for an APK	Evaluators install the APK; no Play Store needed	Free builds are limited per month and wait in a low-priority queue — build the final APK at least 2 days early
Model downloads	Hugging Face Hub	Already used by the pipeline	Pre-download demo models before the day

Everything runs on free tiers; one VM does the heavy work



showcase deployment · free tiers

The browser loads the website from Vercel and both the browser and the phone call the API on the VM over HTTPS; only the worker talks to Hugging Face.

We ruled out Hugging Face Spaces for the backend: Docker and Gradio Spaces now require a paid plan to create, and only static Spaces stay free. Render-style free tiers give about 512 MB RAM, too little for conversion.

Fallback if the cloud VM fails on the day: run the same Docker Compose on your laptop and expose it with a free Cloudflare quick tunnel (`cloudflared tunnel --url http://localhost:8000`), then point the website and app at the tunnel URL. Test this fallback once in week 23.

Demo models

Model	Size (F16)	Use at the showcase
SmolLM2-135M-Instruct	≈ 0.27 GB	Live job from the website (fastest)
SmolLM2-360M-Instruct	≈ 0.7 GB	Live job from the mobile app
Qwen2.5-0.5B-Instruct	≈ 1 GB	Live only if it measures under 3 minutes in week 2; otherwise pre-run
TinyLlama-1.1B, Llama-3.2-1B, Qwen2.5-1.5B, SmolLM2-1.7B	2.2–3.4 GB	Pre-run the day before on the hosted server; shown from History to explain trade-offs and the recommender

Times on the 2-OCPU server are estimates until measured; keep a model in the live list only if a full job (cached download → F16 → Q5_K_M + Q4_K_M → smoke test → benchmark → perplexity → recommendation) finishes in under 3 minutes.

Demo mode is a server setting, shown on every result, so the shortcut is honest:

- Perplexity on 10 WikiText-2 chunks instead of 100.
- Benchmarks with a short prompt and generation (pp128, tg32), 3 runs.
- Demo models' weights pre-downloaded, so live jobs skip the download step.

Showcase script (about 10 minutes)

1. Open the hosted website on the projector; one sentence on the problem.
2. Start a SmolLM2-135M job (both levels, an 8 GB device profile); talk through Live Progress while it runs.
3. Start a SmolLM2-360M job from the phone app at the same time; it queues and then runs.
4. Show a preflight failure (a profile with too little disk) and its message.

5. When the 135M job completes, show the comparison table, the recommendation and its reason; download one GGUF.
6. Open the pre-run Qwen2.5-1.5B job in History: change the device profile and watch the recommendation change.
7. Close with the architecture slide and what the system does at full scale.

Hosting risks: the VM can be reclaimed if idle, free Arm capacity can be unavailable, and EAS builds can queue. Mitigation: create the VM in week 19, keep it in use, build the APK early, and rehearse the laptop-and-tunnel fallback.

Requirement coverage

Every requirement in the 20-page SRS has a week and an owner; use this table to check progress at each milestone.

SRS requirement	What it covers	Week(s)	Owner
REQ-1 to REQ-4	Catalog display, pinned revisions, size limit, device profiles	1–3	Huzaifah, Sibghatullah
REQ-5	Admin catalog editing	14	Huzaifah
REQ-6 to REQ-9	Job choices and preflight checks	3	Huzaifah
REQ-10	202 + job ID; active-job limit	3–4	Huzaifah
REQ-11 to REQ-16	Download, conversion, quantization, smoke test, logs, limits, clean-up, CLI	1–4	Sibghatullah
REQ-17 to REQ-21	Size, tok/s, peak RAM, perplexity, repeats, change vs F16, settings	5–7	Sibghatullah
REQ-22 to REQ-25	Fit rule, quality guard, ranking, baselines	8–9	Sibghatullah
REQ-26	Live status, stage, %, logs	4	Both
REQ-27	Cancel and retry	11–13	Both
REQ-28	Comparison table and charts	5–7	Huzaifah
REQ-29	CSV/JSON export	8–9	Huzaifah
REQ-30	Downloads, history, 7-day expiry, file reuse, storage limit	7, 12–14	Both
REQ-31, REQ-32	Login, roles, own data only	11	Huzaifah

SRS requirement	What it covers	Week(s)	Owner
REQ-33	Mobile app on the same API	15–18	Both
SRS 5.1 Performance	p95 < 500 ms, load < 3 s, job ⩽ 60 min	15	Both
SRS 5.2 Safety	Data folders only, disk reserve, RAM refusal, time limits, licence	3–4, 7, 13	Sibghatullah
SRS 5.3 Security	bcrypt, env secrets, allow-lists, safetensors only, no shell strings	2, 11, 19	Both
SRS 5.4 Quality	95 % reliability, reproducibility, SUS ⩾ 68, 60 % coverage	4, 10, 19–21	Both

Testing plan, risks and fallbacks

Test continuously, not only in Phase 5: each week's work merges only with its tests green.

Test level	What	When
Unit (Vitest)	llama.cpp command building, preflight rules, each recommender rule, size/RAM maths, output parsers	From week 5, every PR
API integration (Vitest + supertest + mongodb-memory-server)	Express routes, validation, auth, ownership	From week 6, every PR
Worker integration	All stages on SmolLM2-135M (fast)	From week 6, weekly
Regression	Every catalog model × every level end to end	Weeks 9, 13, 20, 24
Front end (Vitest + Testing Library)	API client, forms, results formatting	From week 10
Failure scenarios	Unsupported model, low disk, low RAM, failed download, timeout, worker restart	Weeks 7, 15, 21
Performance	5 concurrent users, p95 latency, dashboard load time	Week 19
Usability	5+ classmates, SUS questionnaire	Week 22

Risk	Early sign	Fallback
A model won't convert with the pinned llama.cpp	Weeks 2–3 manual test fails	Drop it; 4 working models is a full catalog
llama.cpp update breaks the pipeline	New release tempts an upgrade	Never upgrade mid-project; if forced, re-run catalog validation (week-long buffer in Phase 5)
Perplexity too slow on CPU	One run > 15 min	Reduce chunk count, record it in results, keep it identical for all models
Disk or RAM too small	Preflight failures on own machine	Keep catalog ⩽ 1.7 B models; Phi-3.5-mini is optional
Worker or WSL2 problems	Worker unstable by week 8	Run the worker loop inside the API process for the demo; the job documents and queue stay the same
Phone can't reach the API	Network errors in the app	Same Wi-Fi, `--host 0.0.0.0`, laptop LAN IP, open port 8000 in the firewall
Schedule slips	A milestone exit check missed	Cut Low-priority items first (charts polish, Docker, PDF report, search/filter), never testing weeks
One member unavailable	—	PROJECT_STATUS.md, DEV_LOG.md and weekly merges mean the other can continue either lanewe